

Article

CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring for Programming Education

Hyunchan Park ¹, Youngpil Kim ^{2,*}, Kyungwoon Lee ^{3,*}, Soonheon Jin ¹, Jinseok Kim ¹, Yan Heo ¹, Gyuho Kim ¹ and Eunhye Kim ¹

¹ Department of Computer Science and Artificial Intelligence, Jeonbuk National University, Jeonju 54896, Republic of Korea; hyunchan.park@jbnu.ac.kr (H.P.); jinseok1006@jbnu.ac.kr (J.K.); gjdhks1212@jbnu.ac.kr (Y.H.); hye030811@jbnu.ac.kr (E.K.)

² Department of Information and Telecommunication Engineering, Incheon National University, Incheon 22012, Republic of Korea

³ School of Electronics Engineering, Kyungpook National University, Daegu 41566, Republic of Korea

* Correspondence: ypkim@inu.ac.kr (Y.K.); kwlee87@knu.ac.kr (K.L.)

Abstract

This paper introduces CodeDive, a web-based programming environment with real-time behavioral tracking designed to enhance student progress assessment and provide timely support for learners, while also addressing the academic integrity challenges posed by Large Language Models (LLMs). Visibility into the student's learning process has become essential for effective pedagogical analysis and personalized feedback, especially in the era where LLMs can generate complete solutions, making it difficult to truly assess student learning and ensure academic integrity based solely on the final outcome. CodeDive provides this process-level transparency by capturing fine-grained events, such as code edits, executions, and pauses, enabling instructors to gain actionable insights for timely student support, analyze learning trajectories, and effectively uphold academic integrity. It operates on a scalable Kubernetes-based cloud architecture, ensuring security and user isolation via containerization and SSO authentication. As a browser-accessible platform, it requires no local installation, simplifying deployment. The system produces a rich data stream of all interaction events for pedagogical analysis. In a Spring 2025 deployment in an Operating Systems course with approximately 100 students, CodeDive captured nearly 25,000 code snapshots and over 4000 execution events with a low overhead. The collected data powered an interactive dashboard visualizing each learner's coding timeline, offering actionable insights for timely student support and a deeper understanding of their problem-solving strategies. By shifting evaluation from the final artifact to the developmental process, CodeDive offers a practical solution for comprehensively assessing student progress and verifying authentic learning in the LLM era. The successful deployment confirms that CodeDive is a stable and valuable tool for maintaining pedagogical transparency and integrity in modern classrooms.

Keywords: Integrated Development Environment; programming education; activity monitoring



Academic Editor: Antonio Visioli

Received: 26 August 2025

Revised: 17 September 2025

Accepted: 22 September 2025

Published: 25 September 2025

Citation: Park, H.; Kim, Y.; Lee, K.; Jin, S.; Kim, J.; Heo, Y.; Kim, G.; Kim, E. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring for Programming Education. *Appl. Sci.* **2025**, *15*, 10403. <https://doi.org/10.3390/app151910403>

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In programming education, the ability to track and analyze students' coding activities in real time is critical for providing effective feedback, supporting individual learning paths, and ensuring fair assessment. This process-level transparency allows instructors

to monitor student progress in detail, identify struggle points early, and offer targeted interventions, fostering deeper engagement and conceptual understanding. Concurrently, with the recent rise of large language models (LLMs), such as ChatGPT (<https://chat.openai.com> (accessed on 22 September 2025)), a new challenge has emerged: students can now generate working code simply by entering natural language prompts, raising serious questions about authorship and the authenticity of submitted work. Studies [1–3] have raised concerns about plagiarism and misuse of LLMs in programming assignments, noting that ChatGPT is increasingly used alongside traditional code-sharing platforms for dishonest behavior. Over-reliance on AI-generated code can also hinder students from engaging in essential trial-and-error learning processes, leading to long-term gaps in conceptual understanding [2].

Concurrently, a persistent challenge in programming education is the prevalence of technical issues arising from inconsistencies in students' local computing environments. Instructors frequently dedicate excessive time to troubleshooting setup problems, which can cause students to miss valuable learning opportunities. In online learning environments, understanding the dynamics of effort regulation [4] and teacher–student interactions [5] is crucial, as both significantly influence student engagement [5] and the development of critical thinking skills [6]. While cloud-based Integrated Development Environments (IDEs) have emerged to mitigate these environmental discrepancies, many existing solutions lack the capability for fine-grained real-time tracking of coding behavior. They often require client-side installations or provide only rudimentary analytics. Recent advancements in educational technology have focused on collecting and analyzing student coding data. Recent studies [7–9] propose that human pose estimation data can be utilized to analyze the physical behavior of online learners. Other studies [10–13] suggest tools for capturing various aspects of the programming process, from code changes to compilation and debugging activities. These tools can be used with education-focused web IDEs [14,15], which excel at capturing fine-grained coding events for pedagogical analysis.

To address these limitations, we present CodeDive, a web-based IDE designed for practical application in university-level programming courses. Built upon a Kubernetes (an open-source system for automating the deployment and management of containerized applications)-based cloud infrastructure [16], CodeDive provides each student with an isolated Linux development environment, where all code and execution logs are securely stored in persistent volumes (PV, a storage unit in Kubernetes that permanently stores data even if the container is deleted). A core component of CodeDive is the eWatcher module, which integrates a kernel-level extended Berkeley Packet Filter (eBPF, a technology for safely running code in the Linux kernel to monitor networks, performance, and more) tracer with a file system monitor to collect real-time data on code edits, compilations, and executions. These data are then chronologically reconstructed and visualized on an interactive timeline, enabling instructors to monitor student progress in detail. Secure access is managed through a proxy-based authentication mechanism. CodeDive has been successfully deployed and evaluated in two large-scale university courses—Operating Systems and Artificial Intelligence—with over 100 students, confirming its robustness and practical effectiveness.

This paper makes the following contributions:

- We introduce CodeDive, a novel web-based IDE that mitigates environment setup challenges while addressing the need for detailed real-time activity tracking in an era of widespread LLM use. It offers a secure, scalable, and user-friendly platform featuring containerized development environments, a browser-based VS Code interface (WebIDE), and the real-time activity tracking system, eWatcher.

- We present eWatcher, a fine-grained monitoring module that captures real-time coding behavior using eBPF and file system tracing [17]. eWatcher allows instructors to review a detailed timeline of code edits, compilations, and execution events, thereby supporting both pedagogical analysis and academic integrity investigations.
- We validate the CodeDive system through its deployment in live university courses, demonstrating its stability, scalability, and potential to enhance programming education. The system's architecture, which integrates container orchestration, persistent storage, and secure access, provides a practical and effective solution for managing programming assignments and analyzing student learning processes.

The remainder of this paper is organized as follows. We review related work in Section 2 and present the architecture of the CodeDive system in Section 3, focusing on key design considerations for security, usability, and scalability. In Section 4, we outline the user workflows for students and instructors. Section 5 details the implementation, covering the Kubernetes-based deployment strategy, the Snapshot and Activity Collector for real-time event logging, and the secure connectivity mechanisms using JWT (JSON Web Tokens, a compact token format for securely transmitting user information in a web environment) authentication and proxy routing. Section 6 evaluates the deployment of CodeDive in a university Operating Systems course. We explain how the system processes and visualizes student programming activity for instructors, present the results of its performance, and highlight key metrics and observed patterns. Finally, we discuss future work in Section 7 and conclude with our findings in Section 8.

2. Related Work

In recent years, various techniques for developing web-based educational IDEs and coding activity tracking systems have been introduced to enhance the quality of programming education by monitoring and analyzing students' learning processes in real time [10–15]. Commercial platforms, such as GitHub Codespaces (<https://docs.github.com/en/codespaces> (accessed on 22 September 2025)), have also adopted web-based development by offering powerful and containerized development environments directly in the browser. However, while these platforms solve the environment setup problem, they generally lack the built-in fine-grained activity tracking and pedagogical analysis features specifically designed for educational settings. In contrast, education-focused systems like Watcher [14] offer a web-based IDE that allows students to code without requiring a local setup while automatically capturing their code edits, executions, compilations, and other events. It records all coding activities as time-stamped logs, enabling instructors to review students' programming behaviors in detail. Similarly, CodeBoard [15] supports programming education through a web browser, allowing students to focus on coding without the need to set up a development environment. It also maintains a history of code snapshots, which instructors can review to gain insight into students' development processes.

Other studies [10–12,18] have focused on tracking coding activities within independent IDEs rather than web-based environments. For instance, the Blackbox project [10] gathers large-scale coding activity data from BlueJ, a popular Java IDE for beginners, by recording students' code snapshots and IDE interactions. These data have been leveraged to analyze students' coding processes and highlight areas for improvement. Similarly, TaskTracker [11] and DevActRec [12], which function within IntelliJ, capture a variety of programming activities, including compilation events and run/debug actions, offering deeper insights into students' coding workflows. These logs enable the reconstruction of the complete sequence of code changes and the examination of students' problem-solving strategies. Additionally, an Eclipse-based activity tracking plugin [18] has been developed to assist instructors in monitoring students' programming exercises in real time. By analyzing

the sequence of code edits and test runs, instructors can identify potentially suspicious behaviors, such as large code pastes or unusually brief development times, which may suggest plagiarism.

ProgSnap2 [13] facilitates the collection of code snapshots and event logs across various educational coding tools and IDEs, akin to the proposed approach. This capability enables researchers and instructors to aggregate data from multiple sources or utilize general analytics tools for deeper insights. More recently, the advent of Large Language Models (LLMs) has introduced a new dimension to programming education support: automated feedback. A notable example is DAFeeD (Direct Automated Feedback Delivery for Student Submissions) [19], a framework that utilizes LLMs to provide direct and formative feedback on the code of students within a Learning Management System (LMS). While systems like DAFeeD excel at analyzing the final code submission to generate pedagogical feedback, they typically do not focus on capturing the real-time step-by-step development process, which is crucial for understanding student problem-solving strategies and behaviors.

Table 1 compares previous studies on the development of web-based educational IDEs and coding activity tracking systems to the proposed technique, CodeDive. First, compared to existing web-based IDE systems such as Watcher and CodeBoard, CodeDive offers much more elaborate tracking information, including snapshots of every coding activity and real-time monitoring. Such detailed tracking information allows instructors to evaluate students more fairly with CodeDive by investigating students' suspicious behaviors. Second, the web-based IDE and container-based PV management of CodeDive provide advantages over existing IDE-based tracking systems such as Blackbox, TaskTracker, DevActRec, and ProgSnap2. As the container image of each student is managed individually in the CodeDive system, students can easily maintain the execution environments consistent with those of other students taking the same programming class. This reduces the time students spend constructing programming environments and prevents instructors from having to address diverse issues arising from different programming environments belonging to individual students. Namely, the strength of CodeDive lies in this holistic system-level approach, providing a robust and mature infrastructure for the complex demands of modern coding education.

Table 1. Comparison of monitoring tools. The symbols 'O' and 'X' indicate that the feature is supported and not supported, respectively.

Title	Web-Based IDE	Code Snapshots	Build Run History	Real-Time Monitoring	Container-Based PV Management	Secure Connectivity
Watcher [14]	O	Partial	O	O	X	Partial
CodeBoard [15]	O	O	Limited	Partial	X	X
Blackbox [10]	X	O	O	O	X	X
TaskTracker [11]	X	O	O	O	X	Partial
Eclipse-based monitoring [18]	X	O	O	O	X	X
DevActRec [12]	X	O	O	O	X	Partial
ProgSnap2 [13]	X	O	X	X	X	X
DAFeeD [19]	X	Partial	X	X	X	O
Github Codespace	O	Partial	Partial	X	X	O
CodeDive	O	O	O	O	O	O

Additional Related Systems. Beyond the aforementioned tools, several other systems have been proposed for web-based programming environments with integrated monitoring: CodeInsights delivers an analytics dashboard that visualizes student code metrics and common compilation errors to aid teaching assistants [20]. TrackThinkDashboard visualizes self-regulated learning patterns—combining code edits and web browsing logs—to reveal problem-solving strategies [21]. Pytutor integrates an online console compiler with software-testing concepts to assess learners’ test-case design and proficiency [22]. A study on student adoption and perceptions of a web-based IDE reports on the deployment, usability, and acceptance of a browser-based environment in large introductory programming courses [23]. Real-time collaborative programming in undergraduate education examines synchronous web-IDE coding activities and their effects on student engagement and skill development [24]. Baker and Brown leverage IDE telemetry to drive learning analytics workflows by capturing fine-grained editing, debugging, and execution events for adaptive feedback in programming courses [25]. Unlike these specialized analytics or IDE extensions, CodeDive uniquely combines a fully featured, container-isolated web IDE with kernel-level eBPF monitoring of every code edit, compilation, and execution—visualized in a unified real-time dashboard on a scalable Kubernetes platform.

3. System Design

This section outlines the overall architecture of CodeDive, including key design considerations and its major components.

3.1. Design Considerations

The CodeDive system was designed with three key objectives in mind: security, usability, and scalability. These objectives were pursued to ensure a safe and uniform development experience for students and enable instructors to monitor coding activities systematically.

Security. Each student is issued with an isolated workspace, which is container-based. A dedicated WebIDE instance prevents inter-user interference and blocks hostile intrusions, crucially providing a sandbox environment that mitigates opportunities for LLM-facilitated collusion or unauthorized code sharing, thereby reinforcing academic integrity. Login is handled by an external Single Sign-On (SSO) service that issues JSON Web Tokens (JWTs); every request is validated on the server side. All traffic is routed through a reverse proxy (an intermediary server that handles external requests and routes them to internal servers, often for security and load balancing), which conceals the container’s IP address and port number from the public network. The use of container isolation, SSO/JWT authentication, and proxy-level obfuscation establishes a multilayer defense against unauthorized access.

Usability. Students are able to commence coding immediately, with no additional software requirements beyond a web browser. WebIDE is built on a Visual Studio Code (<https://code.visualstudio.com/> (accessed on 22 September 2025)) (VS Code) server and provides graphical editing, compilation, and debugging functions, augmented by an auto-save mechanism and a real-time terminal. Each student is provisioned with a separate WebIDE instance for every course they take. Because these instances run in isolated containers, extensions can be installed on demand without affecting others. By supplying the same toolchain consistently, the system eliminates unnecessary setup steps, allowing learners to focus exclusively on programming and genuine problem-solving, a critical aspect in an era where AI tools can inadvertently divert focus from fundamental learning by providing immediate complete solutions.

Scalability. Container orchestration with dynamic resource allocation scales the platform to multiple courses and large student groups. The system automatically generates

and deactivates individual student WebIDE instances, storing source code and snapshots on Persistent Volumes (PVs) to ensure durability in the event of restarts and migrations. This scalability is vital for extending the pedagogical benefits of detailed process tracking and academic integrity measures to a broad user base, even with the widespread adoption of LLMs in education. A primary replica database architecture, strengthened by an in-memory cache, is an effective solution for distributing read traffic and maintaining high availability. This approach ensures efficient use of computing and storage resources. The combination of these measures yields an integrated platform that meets stringent institutional requirements for security, usability, and scalability. The solution unifies a web-based development environment (WebIDE), real-time coding activity tracking (eWatcher), and common infrastructure services (Frontend, Backend, and PVs).

3.2. Architecture

Now, we describe the architecture of CodeDive in general, as well as the roles of its constituent modules and key components, common services, WebIDE, and eWatcher. Figure 1 illustrates their structure and interactions.

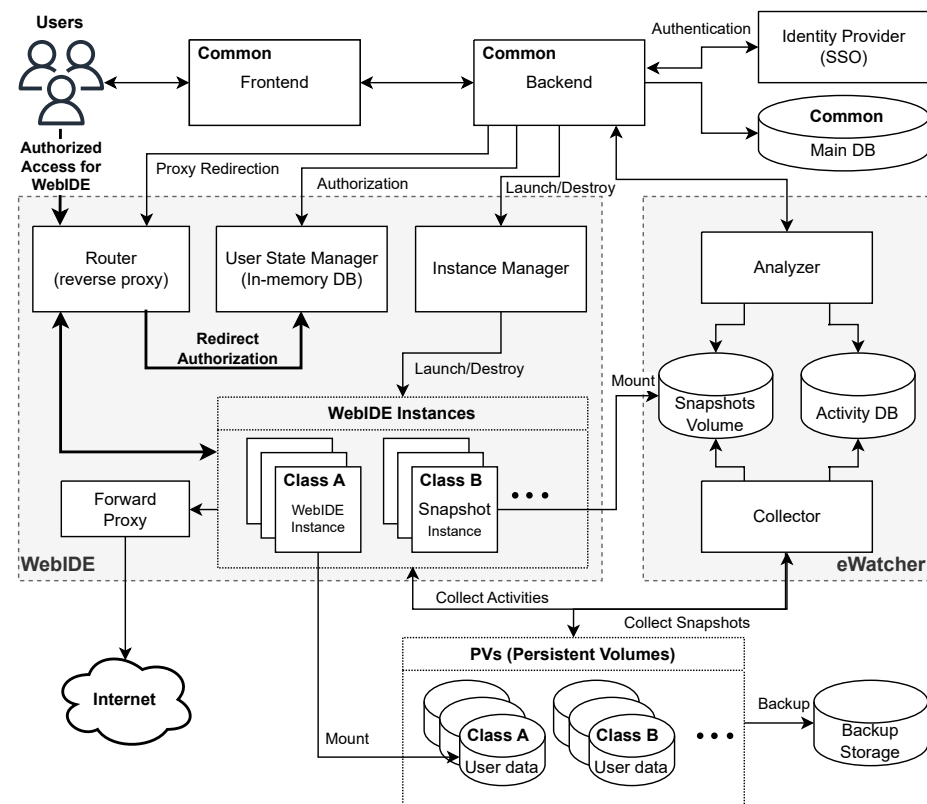


Figure 1. CodeDive architecture.

First of all, user sessions are initiated with external Single Sign-On (SSO, an authentication method that allows access to multiple systems with a single login) authentication. Following identity verification, requests are directed to the CodeDive Frontend and Backend, which in turn route students and instructors to their personal WebIDE environments. The Router, User State Manager, and Instance Manager collaborate to create or terminate WebIDE containers as needed. Concurrently, the eWatcher module, comprising a Collector and an Analyzer, captures and analyzes code edits and execution events in real time. Then, all source files, snapshots, and activity logs are distributed across PVs and a dedicated database. Each WebIDE mounts its PV to store code, settings, and build artifacts. The

eWatcher Collector utilizes this shared storage to ensure complete consistency and integrity in its tracking of coding activity.

During the login process, the CodeDive Frontend initiates authentication, and the Backend then validates the JWT issued by the identity provider and confirms user information in the main database. The Router and User State Manager attach the session to an existing WebIDE container or instruct the Instance Manager to provision a new one. Once the container is online, the Router facilitates traffic between the browser and the instance, ensuring that the internal IP and port data remain concealed. CodeDive delivers a unified development workspace and comprehensive monitoring of learning activities by integrating authentication, container lifecycle management, durable storage, and real-time telemetry. Further sections offer a comprehensive review of each component and its interfaces.

3.3. Key Components

This subsection details the three core components of CodeDive. These components are the foundational Common Services, the WebIDE, which provides isolated development environments, and the eWatcher module for real-time activity tracking. Together, they interact organically to form a stable and scalable educational platform.

3.3.1. Common Services

CodeDive Frontend is a web interface that students and instructors use to log in and out, select lectures and assignments, launch WebIDE, and view analytics dashboards. The interface can be implemented using Single-Page Application (SPA) frameworks such as React or Vue, and all API calls are secured with token-based authentication. A WebIDE launch button appears for each assignment, and an instructor-only dashboard displays eWatcher metrics such as per-student code volume and build history.

CodeDive Backend supervises the system's business logic and authentication/authorization flow. It verifies user tokens issued by the external SSO, looks up user records in the main database, and, through the Router and User State Manager, grants access to the correct WebIDE instance. When it receives coding-activity requests from eWatcher, it queries the database or invokes the Analyzer and returns the results. The Backend is, therefore, the central nexus between the Frontend, eWatcher, and WebIDE.

Persistent Volumes (PVs) provide durable storage for WebIDE instances: all source files, outputs, and snapshot data created by students are stored in the PV. The underlying distributed-storage backend maintains three independent replicas of every block, ensuring that even a disk or node failure does not compromise the data. Even if a container restarts or is rescheduled to another node, the data remain intact, ensuring continuity of service. ReadWriteMany (RWX) mode allows concurrent connections, enabling designated projects to be shared or eWatcher to monitor file changes across all containers. Decoupling storage from containers separates container lifecycle events from data lifecycle guarantees. Per-user volumes enforce strong isolation—no container can read another student's files—while centralizing quota management, capacity expansion, and backup scheduling. Replication, snapshots, and object-storage tiers can be tuned independently of the application stack, optimizing both reliability and cost. Students can close a session and return days later to find their workspace intact, fostering long-term projects and iterative learning. Instructors and administrators gain straightforward backup and restore paths, as well as clear audit trails, while avoiding the risk of data loss common to ephemeral container storage. The net effect is a stable, secure, and scalable foundation that lets the platform focus on teaching rather than disaster recovery.

Main Database (DB) stores user accounts, lecture and assignment data, and role information, and it is accessed directly by the CodeDive Backend. Indexing and caching enable fast responses under heavy load. A primary/read-replica topology supports fault recovery and performance optimization, and backup plus replication policies further strengthen reliability.

3.3.2. WebIDE

WebIDE instances are a Visual Studio Code-based development environment. VS Code-running containers are created on demand for each student and instructor. Each student receives one instance per class they attend, and each professor receives one per class they teach, allowing them to review or prepare assignments directly. By default, once an instance is issued, it remains allocated for the entire semester. Each container manages its own source code, settings, and extensions. When it starts, it mounts the user's PV, allowing code and results from previous sessions to be restored. Because the container image can be updated or rebuilt during maintenance, the PV's role is to preserve user data across such image changes as well as restarts, guaranteeing a consistent work environment.

This per-container architecture delivers three major benefits from a design perspective. First, security and fault isolation: Separation of the OS level prevents the code of one user, malicious or buggy, from interfering with other processes or data. Additionally, network and file system access can be strictly controlled. Second, predictable performance and scalability: each container can be assigned explicit CPU and memory quotas, and a scheduler can distribute instances across multiple hosts, allowing the platform to scale horizontally or burst only during peak hours. Third, operational agility: images can be patched or swapped in a rolling fashion, and a misbehaving container can be recycled without disrupting the wider service; course-specific images or student-requested libraries can be added without risk of dependency conflicts.

Educationally, dedicated WebIDE instances create a safe ownership-driven workspace that encourages experimentation. Students can install tools, test privileged commands, or run long builds without worrying about the impact on their peers, promoting deeper engagement and real-world problem-solving habits. Teachers, meanwhile, gain cleaner assessments: environments are isolated, reproducible, and auditable, reducing opportunities for collusion or accidental interference. In summary, the architecture combines enterprise-grade robustness with learner-centric flexibility, making it well-suited for modern programming instruction.

Router is the proxy module of CodeDive. When a user requests access to WebIDE, Router checks the user's JWT and forwards the traffic to the appropriate WebIDE instance. This approach hides the container's actual IP and port, allowing administrators to manage multiple instances under a single domain. Placing a proxy at the edge creates a clear security boundary. External clients can never access containers directly; all access control, Transport Layer Security (TLS) termination, and firewall rules are enforced once, at the Router. Because the routing table updates in real time, new WebIDE instances become reachable the moment they start, and stale entries are removed as soon as a session ends, preventing orphaned routes. The same component can apply per-tenant hostname prefixes, CPU- or memory-based admission rules, and even spread the load across multiple Router replicas, enabling horizontal scaling without exposing the internal topology. For students, the Router delivers consistent connection semantics: clicking the same "Open WebIDE" button in the Frontend always opens their personal IDE, regardless of maintenance, migration, or scaling events. Instructors and administrators gain a single vantage point for logging, rate limiting, and fine-grained access policies, which simplifies auditing and incident response.

Multi-institution deployments can partition traffic by domain while still sharing core infrastructure, supporting diverse courses under one secure centrally managed gateway.

User State Manager keeps user-session status and container-allocation data in memory for quick lookup. Working with Router, it manages redirect authorization and tracks each user's WebIDE identifiers and access paths. It can therefore complete tasks such as login, logout, and permission checking rapidly and synchronizes with the CodeDive Backend when necessary to maintain a consistent state. Centralizing session logic enhances security and coherence: all login, logout, and permission checks are consolidated through a single trusted point, thereby eliminating scattered states. Real-time lookups enable the platform to react quickly to policy changes—such as concurrent login limits or forced expirations—without affecting other components. The module's single-responsibility design also makes it easy to adjust timeout rules or concurrency caps without modifying the rest of the stack.

Instance Manager controls the lifecycle of WebIDE instances: it creates a container if none exists when a user opens an assignment, and it terminates idle containers to reclaim resources. This policy optimizes resource utilization and enables automatic scaling when a large number of users are active. By isolating execution control from policy decisions, the module can swap orchestration backends—Docker, Kubernetes, or future runtimes—without altering higher layers. It enforces per-container CPU and memory quotas, supports multiple image types (e.g., GPU-enabled classes), and queues burst requests so that large cohorts can start smoothly.

3.3.3. eWatcher

Collector records coding events, such as file modifications, builds, and executions, inside each WebIDE instance. It gathers these logs in real-time or at short intervals, structures the data with details like file names and commands, and then passes the structured data to the Analyzer. Separating collection from analysis keeps the pipeline fast and resilient. The Collector's single job is to accept and package events; heavy computation lives elsewhere, so bursts of activity cannot block ingestion. Multiple Collector instances can be deployed for horizontal scale, while the queue decouples their throughput from that of downstream processors. This loose coupling localizes failures—an Analyzer slowdown never causes data loss, and a Collector restart leaves previously queued data intact. For administrators, this architecture provides high-fidelity telemetry without taxing user containers and allows them to replay queued data after maintenance with no gaps. For instructors, this continuous stream of granular data empowers them to conduct time-series analyses of learning patterns, identify struggle points early, and make real-time interventions (e.g., offering hints after repeated compile errors), providing an invaluable transparent view into student effort that is often obscured by LLM-generated submissions, thereby enabling authentic assessment. In short, Collector provides the reliable low-latency data foundation essential to make data-driven programming education a reality.

Analyzer performs an in-depth examination of the coding events and snapshot data. It can aggregate, for example, a student's total workload, edit frequency, or build success rate over time, and it can detect anomalies such as large amounts of code produced in an unusually short period. The results are stored in eWatcher's databases (see Databases subsection) and can be visualized through the CodeDive Backend or the instructor dashboard. Isolating analysis from collection decouples computing-intensive work from real-time ingestion. The Analyzer can execute batch jobs, statistical pipelines, or machine-learning models without risking data loss upstream. Because it is a stand-alone service, new algorithms or external BI connectors can be added without changes to the IDE layer; the module simply consumes queue data and publishes enriched results. Horizontal scaling is straightforward—more

Analyzer workers mean faster processing, while the Collector continues buffering events at line rate. For instructors, the Analyzer surfaces class-wide trends (common errors, idle periods) and student-level indicators (persistence, iteration depth, possible plagiarism, or unusual coding patterns indicative of LLM over-reliance), enabling timely intervention and fair assessment in the AI-assisted coding era. Learners receive data-driven feedback on their habits and progress, fostering self-regulated improvement. Administrators gain system-health signals—spikes in compile times or off-peak usage—that guide capacity planning and feature tuning. In short, the Analyzer is the platform’s reasoning engine, converting high-volume telemetry into insights that enhance both teaching effectiveness and operational excellence.

Databases The eWatcher module utilizes two specialized databases to store and manage the collected data: the Activity DB and the Snapshots DB. The Activity DB stores structured records of all coding events, such as work time, commands used, execution results, and file-size changes. These structured data are optimized for efficient statistical analysis and serving queries for the instructor dashboard. The Snapshots DB maintains the history of all code snapshots. For every file change detected by the Collector, a complete copy of the source file is recorded along with its metadata, including its path and version. Although the actual files reside on the Persistent Volume (PV), this database stores the associated metadata, allowing the restoration of any previous state or a detailed examination of code evolution.

Separating these databases from the Main DB offers three key advantages. First, it protects performance by allowing each database to be tuned for its specific workload: the Main DB is indexed for low-latency queries, while the Activity and Snapshots stores are optimized for high-volume append-only writes. Second, this design enhances scalability and cost-efficiency, as both databases can be sharded or scaled out independently, and large objects like snapshot blobs can be moved to economical object storage. Finally, the physical separation limits the blast radius of a potential privacy breach, as the auxiliary stores contain only anonymized activity data.

Snapshots let users and instructors roll back to any prior state, review code progression, and recover from accidental loss. Activity logs fuel the Analyzer’s metrics and help teachers pinpoint widespread misconceptions or individual areas of struggle. Administrators gain an audit trail for postmortems and capacity planning: sudden surges in compile errors or after-hours usage patterns become visible without burdening the production database. In short, these dual stores turn raw chronicles into a durable asset for learning analytics, user support, and system reliability.

4. Workflows

In this section, we will describe in detail the execution flow and functionalities that occur when using CodeDive for actual users, such as students and professors. The overall workflows for students and professors are illustrated in Figure 2a,b, respectively.

4.1. Students

(1) Portal login and initial access

Students are required to access the CodeDive portal (Frontend) via a web browser and enter their account information on the login screen linked to their school SSO or external identity provider. The subsequent login request is directed to the CodeDive Backend, where the authentication token issued by the external SSO is verified. If verification is successful, the token is then used to retrieve the student’s profile information from the primary database. Consequently, the student is granted access

to the portal dashboard, where they can view registered courses and assignments, and employ features such as “Open WebIDE”.

(2) **Connecting to Personal Workspace and Configuring Environment**

When a student selects a particular assignment from the dashboard and clicks the “Open WebIDE” button, the CodeDive Backend performs a series of actions. Firstly, it checks the User State Manager (In-memory DB) to ascertain whether the student’s WebIDE container (workspace) is already operational. If a container is unavailable, the Instance Manager generates a new one from a standardized image. This image includes development tools, a VS Code server, and the eWatcher client. At this stage, the Persistent Volume (PV) is mounted automatically to facilitate the restoration of the source code and settings from the previous session. When the container is initiated, the CodeDive Router establishes a proxy connection between the student’s browser and the designated WebIDE instance. This feature enables the immediate use of the web-based VS Code editing interface, eliminating the need for any additional local development tool installation.

(3) **Coding, Execution, and Activity Tracking**

Students will acquire the skills to write and edit code in the WebIDE and to execute compilation or execution commands via the terminal window. WebIDE is equipped with an auto-save feature, ensuring that any modifications made to files are seamlessly reflected in the container’s internal file system. For instance, when a student edits and saves the “main.py” file, the time of the modification is recorded. If there is no explicit save command, the system will automatically generate a snapshot of the file’s content if the user pauses typing for over one second after a file modification. This threshold was chosen by referencing the default 1000 ms *afterDelay* setting in Visual Studio Code’s widely used Auto Save feature, which represents a well-established balance between capturing granular changes and avoiding excessive system load. Subsequently, upon selecting the “Run/Debug” button or invoking the Python main.py function in the terminal (<https://www.python.org/>), the eWatcher module (Collector and Analyzer) within the container gathers information regarding the initial and final times of the process, as well as the execution command. The collected events and metadata are then transmitted to the eWatcher Backend, where they are recorded in the Activity DB and Snapshots DB. These databases can then be used for subsequent analysis and visualization tasks.

(4) **Saving and Ending the Session**

All alterations and execution histories that transpire while the student continues coding are accumulated in real time in the database and PV through the eWatcher module. Following the termination of the task or in the event of a period of inactivity, the CodeDive Backend has the capability to initiate a stoppage of the container in accordance with the established idle policy. Note that containers shall remain operational throughout the semester unless system resources become constrained. The codes stored in the PV are permanently preserved, ensuring that when a student reconnects, the previous work environment is restored, enabling continuous work. Consequently, students can code in a consistent and stable development environment without the need for additional settings.

4.2. Professors

Professors primarily use two main features of the CodeDive system. First, they can access students’ WebIDE instances to check their progress and provide hands-on help with coding. Second, the platform allows them to monitor and analyze students’ coding

activities using the eWatcher tool. These features can also be used by teaching assistants assigned by the professor. The following section explains how professors utilize eWatcher.

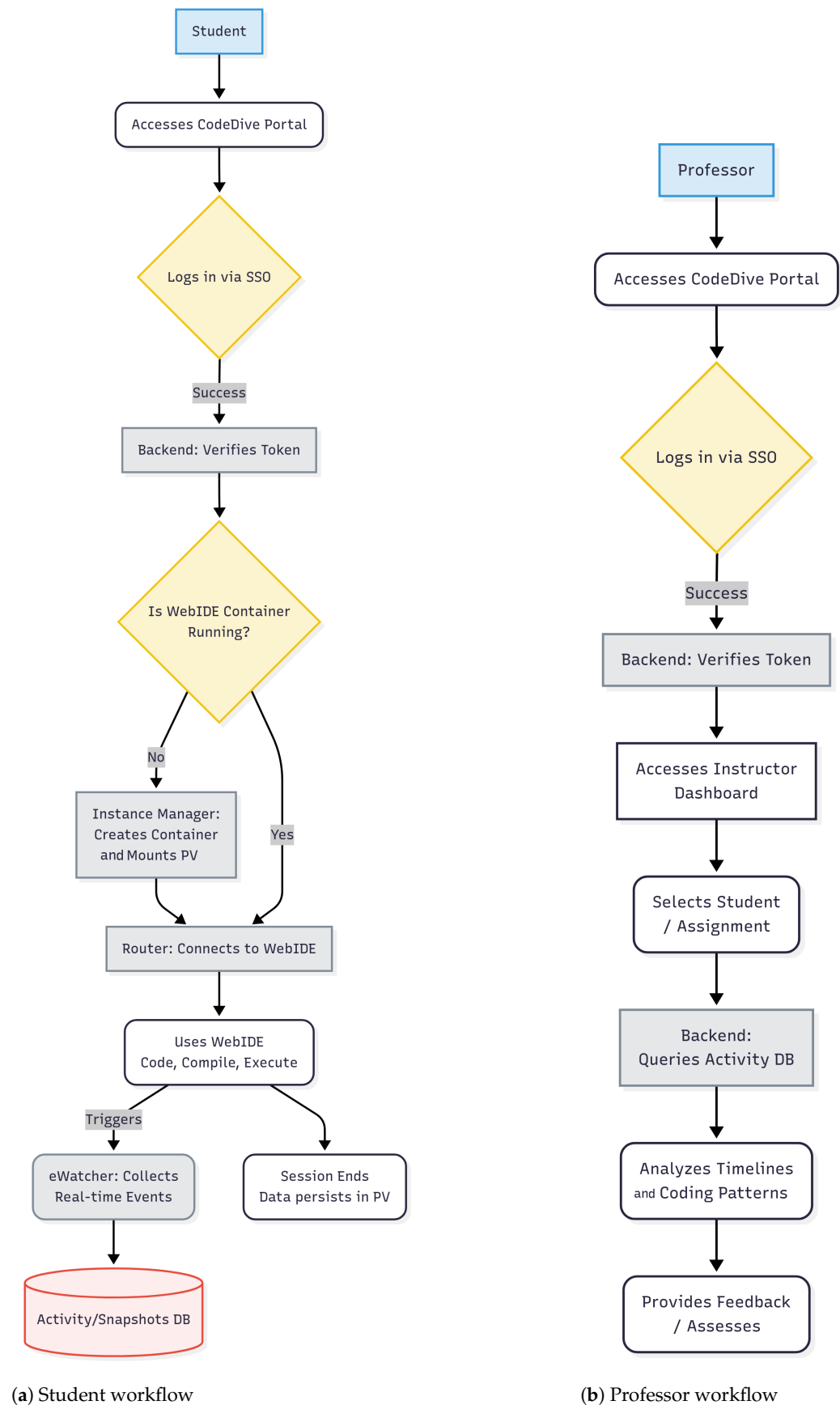


Figure 2. Overall user workflows in CodeDive for students and professors.

(1) **Professor Portal Login and Dashboard Access**

Professors are required to log in to the CodeDive portal with their accounts. Privileges are verified through external SSO. Upon successful authentication, CodeDive Backend facilitates the activation of the professor-only management dashboard. This dashboard provides an interface for professors to view a comprehensive list of their courses, registered assignments, and student participation status for each assignment.

(2) **Observation of Student Activities and Comprehensive Record Analysis**

From the dashboard, faculty members can select specific courses or assignments to access statistics about the coding activities of participating students. The CodeDive Backend queries the eWatcher Backend and Activity DB to aggregate summary data, including the number of edits, build and execution frequency, and last login time for each student in real time. The CodeDive Backend then visualizes this data in tables or graphs. When a specific student is selected, their detailed coding logs are provided in chronological order, along with information such as the coding timeline, file size changes, and compilation and execution results. This affords professors the opportunity to meticulously analyze the patterns exhibited by students during task execution, as well as to observe the evolution of their problem-solving methodologies.

(3) **Plagiarism detection and feedback**

Professors employ a comparative and analytical approach to assess work patterns across a cohort of students. This assessment is informed by the aggregation of coding activity logs and the subsequent visualization of data. For instance, if the majority of students code at various times, but one student uploads a substantial amount of code in a brief period just before the deadline, this could be indicative of potential plagiarism. By leveraging coding activities, the CodeDive system provides a framework for detecting such anomalies. This empowers professors to conduct further verification or offer individualized guidance based on substantiated evidence. Furthermore, professors have the ability to employ the statistics and graphs provided on the dashboard to offer specific feedback on individual students' coding habits and, when necessary, export data in CSV or report formats for utilization in course management and educational quality improvement.

5. Implementation Details

The implementation of CodeDive spans three core areas: (1) a Kubernetes-based deployment architecture that delivers elastic scaling, high availability, and secure multi-tenant isolation; (2) a Snapshot and Activity Collector subsystem that captures both file-system and process-level events in real time; and (3) an end-to-end secure connectivity layer employing JWT authentication and proxy routing to protect user sessions. In the subsections that follow, we first describe how CodeDive is orchestrated within a Kubernetes cluster and the benefits this provides in educational settings. We then detail the mechanisms by which code edits, builds, and executions are instrumented and logged. Finally, we explain how user credentials and network traffic are managed to ensure that each WebIDE instance remains both accessible to its owner and hidden from unauthorized actors.

5.1. Kubernetes-Based Deployment

This section outlines the deployment of the CodeDive system within a Kubernetes cluster, highlighting the key advantages tailored to educational environments. These advantages include automatic scaling, high availability, enhanced security, and data consistency.

5.1.1. Deployment Method

The CodeDive system has been developed to operate all services within a Kubernetes cluster environment. The system's components, including the CodeDive Frontend, Backend, WebIDE Container Instances, and eWatcher module, are deployed as distinct pods and managed using YAML manifests or Helm charts [26]. Specifically, WebIDE instances are created and terminated by the Instance Manager via Kubernetes API calls, and each container automatically mounts a PV to securely store user data. In addition, the eWatcher module has been implemented as a DaemonSet (a Kubernetes controller that ensures a copy of a specific Pod (group of containers) runs on every node in the cluster), which is now active across all nodes in the cluster. This ensures that file and process events occurring on each node are collected without omission. The Ingress Controller and NetworkPolicy are responsible for managing external access, while JWT authentication is used for routing and traffic distribution. This deployment method is designed to respond flexibly to changes in demand by automatically adjusting the number of containers and resources through Kubernetes' scheduler and autoscaling features.

5.1.2. Key Advantages

The use of Kubernetes-based deployment provides a broad range of specialized benefits for the CodeDive system. Firstly, the auto-scaling feature automatically creates additional nodes and pods, ensuring efficient distribution of load even in cases of a rapid increase in the number of students. This facilitates the dynamic creation of student-specific WebIDE instances and the Instance Manager's role, ensuring the continuity of the learning environment. Additionally, Kubernetes' high availability features enable automatic redistribution of containers to other nodes in the event of a node failure, ensuring uninterrupted service operation. The implementation of rolling updates and zero-downtime deployment capabilities serves to mitigate the impact on users during system component version upgrades or patch applications. Furthermore, Kubernetes strictly controls communication between WebIDE Instance, eWatcher, and Backend through namespace separation and NetworkPolicy-based security policies, significantly reducing the risk of external attacks or internal information leaks, thereby bolstering academic integrity against various forms of misuse, including those related to LLM-generated code, by ensuring a secure and controlled environment. The RWX mode of PVs enables the concurrent use of the same storage by multiple containers, ensuring data consistency and continuity even in scenarios where containers are restarted or relocated to different nodes. In addition to these foundational benefits, Kubernetes further enhances system integrity through a multi-layered security model. To prevent container escapes, all pods run under a restricted Security Context, which applies seccomp (a Linux kernel security feature that prevents containers from making dangerous system calls) profiles and enforces non-root user privileges to block potential attacks on the host system. Furthermore, user data isolation is guaranteed at the storage layer. Each student is assigned a unique Persistent Volume Claim (PVC) (a Kubernetes object used by a user to request storage space from a PV), and Role-Based Access Control (RBAC) (a method of controlling access to system resources based on the roles of users or services) policies ensure that a container can only mount its specifically authorized volume, making it architecturally impossible to access another student's data, which is fundamental for fair assessment and preventing collusion, especially when verifying originality against potential LLM-assisted collaboration. These granular security controls are key elements that heighten the overall reliability of the CodeDive platform.

5.2. Snapshot and Activity Collector

CodeDive ensures that student code, build outputs, and other work data persist securely. We use PVs for durable storage. A real-time monitoring system collects snapshots and coding activity logs to form a comprehensive record of activity. Dynamic WebIDE instances mount PVs in RWX mode, enabling precise change detection.

Each WebIDE instance is created independently for each student and mounts a unique working directory on the PV to continuously store source code and outputs. The storage system has been configured to enable multiple containers to access it concurrently, thereby ensuring data consistency while preserving scalability. When a student saves a file, the container module reads it from the PV to capture the latest state. It then compares the file's new size to the previous snapshot. If sizes differ, it splits the file into fixed-size segments (e.g., 8 KB) to avoid duplicate events. When it detects a change, it copies the full file to the snapshot volume. Concurrently, metadata such as the file path, size, and save time are generated. These metadata are transmitted to the eWatcher Backend via REST API (a standardized set of rules for exchanging information between web services) and used as basic data for subsequent analysis and visualization tasks. This configuration is instrumental in preserving the integrity of the original data while enhancing network bandwidth efficiency.

We collect student coding activities using two methods. First, the system captures process start and stop events. To accomplish this objective, the eWatcher-proc module employs an eBPF program in the Linux kernel to monitor all process events occurring in the WebIDE instance in real-time. eBPF injects secure bytecode into the kernel, facilitating not only network packet filtering but also the monitoring of various kernel events. Consequently, this capability enables precise recording of events from processes that execute for brief periods, which is crucial for capturing the detailed step-by-step programming behaviors needed to differentiate genuine student work from LLM-generated solutions that often appear as sudden complete code insertions or final products without a discernible iterative process. Specifically, this monitoring function is implemented on worker nodes within a Kubernetes cluster, facilitating the expeditious collection of events occurring in WebIDE containers dispersed across each node. It uses multiple small handlers. Upon detection of the `sched_process_exec` event by the initial handler, the `task_struct` in the kernel is accessed to collect the information necessary for container identification. Subsequent handlers then collect additional information, such as the current working directory and command-line arguments, storing the process ID as the key in a BPF hash map. In the event of a process termination, the stored metadata are retrieved to construct information such as the executable file path, command-line arguments, exit code, and the time of occurrence. This information is then transmitted to the eWatcher Backend via a REST API. This methodology facilitates the precise documentation of events from ephemeral processes, ensuring the absence of omissions.

Second, the eWatcher-code module uses `inotify` and Python's `watchdog` library to monitor PV-mounted working directories in real time, detecting file open, write, and close events [27,28]. This fine-grained file system monitoring is essential for reconstructing the natural iterative development process, which stands in stark contrast to the often instantaneous or large-batch code submissions characteristic of LLM usage. This module has been designed to manage directories that are created or deleted dynamically through a process of recursive monitoring. It employs a comparison of file sizes to ascertain the extent of alterations among multiple modification events. Additionally, it utilizes a debounce technique to divide and compare file contents in fixed units as needed. Furthermore, it guarantees that events within the same directory are processed sequentially by leveraging Python's `asyncio` framework and event queue, thereby ensuring data reliability. Finally,

metadata such as the file path, size, and save time, along with snapshots of modified files, are transmitted to the eWatcher Backend, enabling the construction of precise coding activity logs.

5.3. Ensuring Secure Connectivity

CodeDive employs JWT tokens and a reverse proxy to guarantee secure access to WebIDE instances. This configuration enables users to securely access the WebIDE Instance. It verifies each user's credentials and hides container details behind the proxy.

5.3.1. JWT-Based Authentication in the Frontend

The CodeDive Frontend issues a JWT token from an external SSO (such as Keycloak [29]) after the user successfully completes the integrated login process using their school email address. The issued token undergoes verification by the CodeDive Backend and is subsequently securely stored in the client-side session storage. This JWT token is incorporated into all API calls and WebIDE connection requests and is transmitted over HTTPS in an encrypted format. The Frontend checks each token's expiration and automatically refreshes it five minutes before it lapses. This ensures that the most recent authentication information is consistently maintained. This process enhances the reliability of user authentication and minimizes security threats that may arise during communication with the Backend.

5.3.2. CodeDive Router-Based Proxy Connection

In the event that a user requests WebIDE access, the Backend verifies the validity of the JWT. This analysis is based on the user identification information and role information contained in the token. The CodeDive Backend collaborates with the User State Manager to determine the universally unique identifier (UUID) and access path of the WebIDE Instance that the user can access. The CodeDive Router subsequently establishes HTTP and WebSocket connections in order to configure a proxy connection between the user's browser and the WebIDE Instance. The container's IP and port remain concealed behind the reverse proxy domain. During the process of establishing a proxy connection, the JWT token and UUID information undergo re-verification to effectively impede unauthorized access or attempts at token tampering. To verify JWT signatures, we utilize RSA or ECDSA public keys, which are commonly used for server authentication in secure protocols like HTTPS. A cache-based blacklist is a system that blocks revoked or reused tokens, thus preventing replay and forgery. Kubernetes deployment provides the real-time processing, security, and scalability that CodeDive requires for large-scale use. This deployment serves as a fundamental technical foundation, enabling real-time data processing, security, and scalability—all of which are crucial in educational environments.

6. Implementation Results

In this section, we present our experiences deploying CodeDive in a real classroom setting and discuss the insights gained from the collected data. We first outline how CodeDive structures and processes learners' programming activity logs. We then introduce the instructor-facing user interface components that visualize these analytics in real time, demonstrating how the collected data can be used to monitor and guide student progress. Next, we describe the results from a large undergraduate Operating Systems course where CodeDive was used to monitor Assignment 1, highlighting the key metrics captured and the patterns observed. Finally, we evaluate the system's stability and resource overhead during this deployment.

6.1. Log and Analytics

6.1.1. Log Architecture

CodeDive persists every student action in three tables—Snapshot, Build, and Execution—whose fields are defined in Table 2. Each record contains the learner’s `student_id`, the `class_div`, and the `hw_name`, enabling precise tying of events to individuals, course sections, and assignments.

The Snapshot table captures each source-file modification by logging the file name, new file size, and timestamp. The time series assembly of these snapshots reveals coding volume fluctuations, identifying whether a student works in concentrated bursts or intermittently, which in turn reflects study habits and focus patterns. For example, sustained coding streaks may indicate periods of deep focus (or last-minute effort), whereas sporadic smaller changes suggest a more continuous and incremental approach. These snapshot trends allow instructors to infer each student’s working style—whether they tend to code in intensive bursts or maintain steady progress over time—providing clues about their study habits and time management.

Table 2. Field definitions for Snapshot, Build, and Execution. The ✓ symbol indicates that a field is applicable to the corresponding category.

Field	Description	Snapshot	Build	Execution
<code>id</code>	Record unique identifier	✓	✓	✓
<code>class_div</code>	Course section/division	✓	✓	✓
<code>hw_name</code>	Assignment name	✓	✓	✓
<code>student_id</code>	Student number	✓	✓	✓
<code>filename</code>	Source file name	✓	—	—
<code>cwd</code>	Working directory	—	✓	✓
<code>file_size</code>	File size (bytes)	✓	—	—
<code>binary_path</code>	Compiler executable path	—	✓	—
<code>process_type</code>	Execution type (e.g. python, java)	—	—	✓
<code>cmdline</code>	Command-line string	—	✓	✓
<code>exit_code</code>	Return code	—	✓	✓
<code>target_path</code>	Target file path	—	✓	✓
<code>timestamp</code>	Event timestamp	✓	✓	✓

The Build table records every compilation attempt, including the working directory, the build command, the compiler path, the output target, the exit code, and the timestamp. Analysis of these data illuminates students’ debugging proficiency and assignment completion quality through success rates and error patterns. For instance, a student with many consecutive failed compilations before a successful build is likely encountering and fixing multiple errors (trial-and-error debugging), whereas a high success rate in builds may indicate that the student writes code with fewer mistakes or tests thoroughly between attempts.

The Execution table documents all program executions by logging the execution command, return code, working directory, process type, target file path, and timestamp. By linking executions with their preceding snapshots and builds, CodeDive reconstructs each learner’s complete edit–build–run cycle, allowing evaluators to inspect the entire development process rather than only the final deliverables. This holistic view means that instructors can see not only the final output of a student’s work but also how the student arrived at it. For example, one can observe how frequently each student runs their program (indicating how often they test their code), how soon after editing they attempt to execute, and how they deal with runtime errors (via the recorded return codes). Such

context reveals each learner's coding strategy—whether they test continuously or only after major edits—that would be hidden if one looked only at the final deliverables.

6.1.2. Analytics Services

CodeDive exposes three REST-API endpoints that return JSON results for seamless integration into dashboards or external tools. The first endpoint groups Snapshot records into fixed intervals (e.g., five-minute or one-hour bins), sums file sizes within each interval, and computes differences between consecutive intervals to produce a time series of coding activity. This reveals when and how intensively each student is coding, making it easy to spot periods of concentrated effort or inactivity. The second endpoint extracts Build and Execution events per learner, joins each event with the most recent snapshot, and measures the frequency and timing of edit-build-run cycles to reveal individual debugging workflows. For example, a tight loop of frequent edit/build/run events indicates rapid iterative testing, whereas long gaps between builds or executions might suggest the student is spending extended time debugging or planning before running their code again. The third endpoint aggregates the absolute changes in file size throughout the assignment period to calculate the total code variation for each learner, while also calculating secondary metrics: average code volume per interval, mean snapshot count, total build/run counts, and code size percentiles. These aggregate measures provide a multifaceted profile of engagement and progress: for instance, a high total code delta coupled with many builds may signal a student who explored multiple solutions or had to significantly refactor their code, whereas a low code delta with few snapshots could indicate either an efficient implementation or minimal effort. By combining time-series activity data, cycle frequency, and cumulative code changes, CodeDive's analytics give instructors a comprehensive view of both the process and output of each student's work.

6.2. User Interfaces

This subsection presents the primary user interfaces in CodeDive, explains their purposes, and demonstrates how they support course management, coding activities, and analytics. These interfaces provide seamless navigation between the IDE, real-time monitoring, and administrative controls.

6.2.1. Administrator Dashboard

Figure 3 illustrates the instructor and administrator interface in CodeDive. From the top navigation bar, users can switch between the WebIDE, eWatcher analytics dashboard, and various administration panels. In the "Class Management" view, a live table lists all courses currently registered in the system. Each row displays the class name, shorthand code, professor in charge, academic year, term, and section division, alongside icons for editing or deleting the entry. A global search box at the top allows users to filter by class name, course code, or instructor, facilitating efficient course administration. This streamlined interface reduces the administrative overhead for staff managing multiple classes, ensuring that setting up or updating course information in CodeDive is straightforward and efficient. By centralizing class setup and user management, the platform frees instructors to focus on monitoring student progress rather than spending time on configuration tasks.

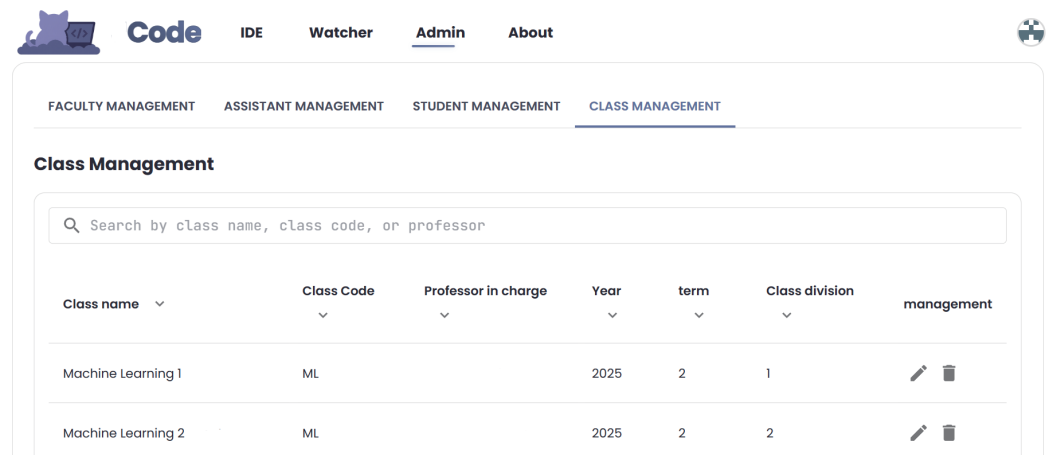


Figure 3. Administrator dashboard for managing courses, users, and assignments. The top navigation bar provides access to the IDE, eWatcher analytics, Admin settings, and account controls. The main panel under “Class Management” displays a searchable sortable list of active courses, showing each class’s name, code, instructor, year, term, and section, with inline edit and delete buttons.

6.2.2. WebIDE Interface

Figure 4 demonstrates the CodeDive WebIDE, which mirrors the familiar Visual Studio Code user interface in a web browser. The left panel contains the project explorer listing multiple assignment directories. The central editor area displays code files with full syntax highlighting and auto-completion support. The bottom pane provides a fully functional terminal where students can compile code (e.g., gcc main.c), run executables (e.g., ./a.out), or install libraries (e.g., sudo apt install python3-matplotlib). By integrating all editing and command-line operations in one browser window, WebIDE removes the need for separate SSH connections or local environment setup, allowing learners to focus solely on coding and experimentation. This unified environment gives all students a consistent development setup and removes common technical hurdles (such as tool installation or remote-access issues).

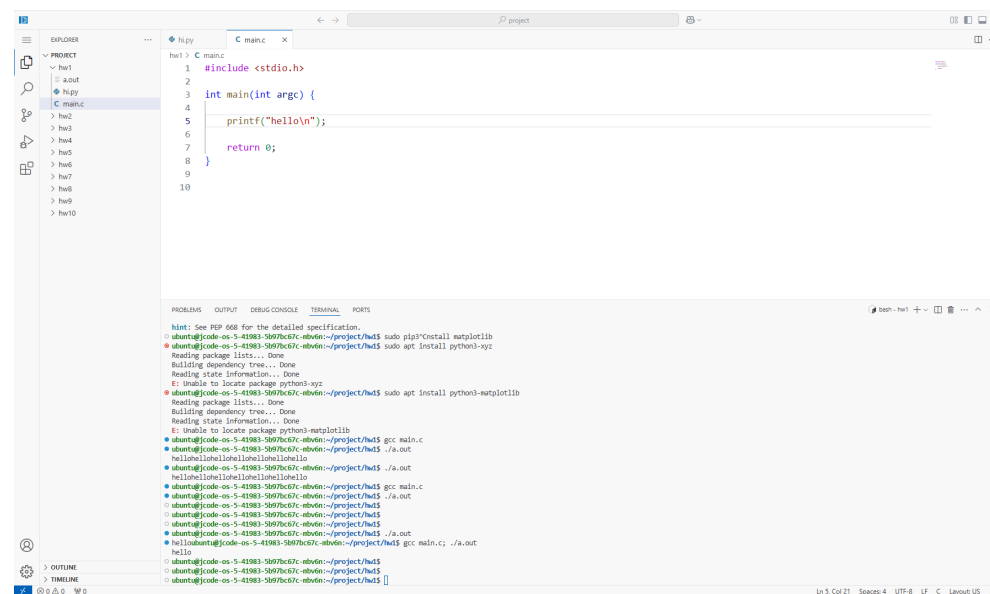


Figure 4. Browser-based IDE replicating the Visual Studio Code environment. The left pane is the file explorer with assignment folders, the central area hosts editable source files with syntax highlighting, and the bottom pane is an integrated terminal supporting compilation, execution, and package installation—eliminating the need for a separate SSH client or local toolchain.

6.2.3. Cumulative Code-Change Chart

As illustrated in Figure 5, each student's total coding activity for Assignment 1 is displayed, quantified as the cumulative code-change volume (insertions plus deletions). The bars are sorted in descending order, with students who wrote or modified more code positioned on the left. By sharing this anonymized visual with the class, we offer learners insight into their peers' effort levels, which can motivate early engagement and sustained progress. Instructors can also use this overview to identify students whose activity falls well below the class average and who may benefit from additional guidance before the deadline. In essence, the cumulative code delta serves as an indicator of each student's overall effort and iteration on the assignment. Students with substantially higher code-change volumes likely underwent more extensive development and debugging (or multiple revisions), whereas those with very low volumes may have either completed the task with minimal trial-and-error or potentially under-engaged. By sharing this chart, students can contextualize their own coding practice relative to their peers in an anonymized non-judgmental way.

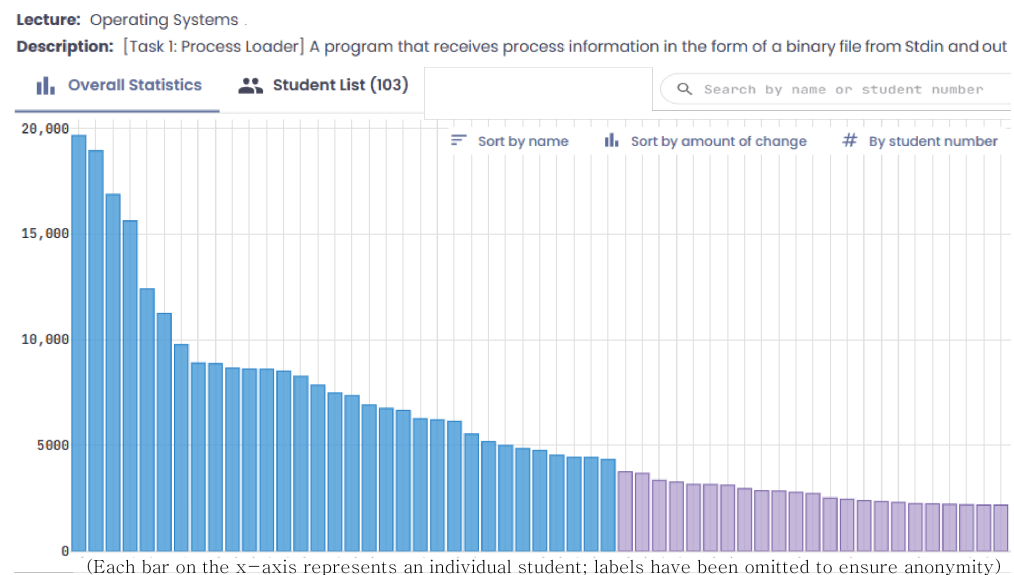


Figure 5. Total code-change volume per student for Assignment 1, measured as the sum of insertions and deletions recorded in the Snapshot table over the assignment period. Blue bars indicate students whose cumulative code delta exceeds the class average; purple bars indicate those below average.

6.2.4. Activity Timeline

Figure 6 visualizes a single student's complete edit–build–run history during one coding session. The solid curve tracks the evolution of the code size, rising from approximately 2200 B to over 3000 B as the student adds functionality. An early orange diamond just before 13:00 marks a compilation failure, followed by a series of blue diamonds indicating successful builds after code edits. At approximately 15:26, the code size peaks at 2628 B, and immediately thereafter, green circles appear, indicating successful executions. A vertical dashed line highlights the selected execution event on 28 April 2025 at 15:27:15. This interactive timeline enables instructors to assess students' development workflows—error frequency, build cycles, and testing patterns—without manually parsing raw log files. For example, an instructor can instantly tell if a student encountered many compilation errors (clusters of orange diamonds) and how quickly those errors were resolved, or if the student waited until a large code increase (a jump in the solid line) before running the program. Such patterns—whether rapid fix-and-run cycles or prolonged gaps due to debugging—are immediately visible on the timeline, enabling timely intervention and targeted feedback.

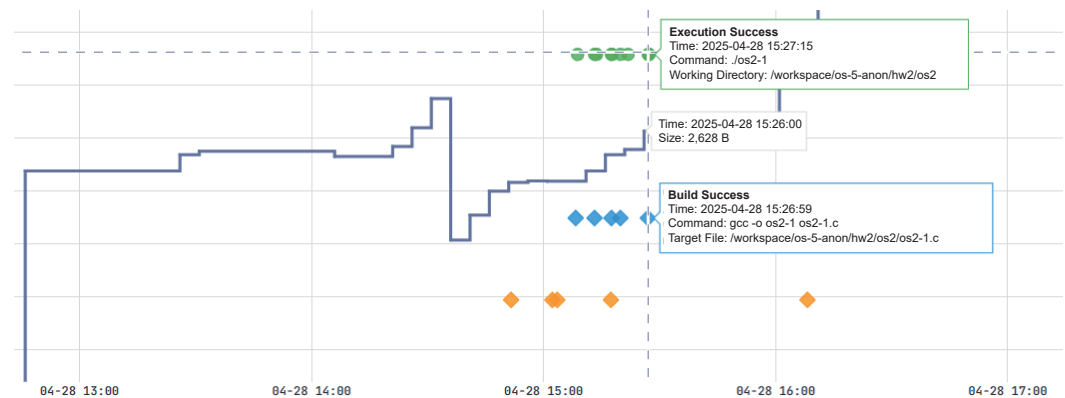


Figure 6. Example of the CodeDive activity timeline. The solid line plots the student’s source code size (in bytes) on the right-hand axis over time. Orange diamonds denote build failures (compiler errors), blue diamonds denote successful builds, and green circles denote successful executions. Hovering over any marker reveals a tooltip with detailed information: the exact timestamp, the full command line (e.g., `gcc -o os2-1 os2-1.c` or `./os2-1`), and the working directory path (e.g., `/workspace/os-5-blank/hw2/os2`).

6.3. Classroom Deployment

During Spring 2025, CodeDive was integrated into the Operating Systems course to collect real-time data for Assignment 1 from 24 March to 4 April. Of approximately 100 enrolled students, 95 performed at least one snapshot, build, or execution. This high adoption rate (95%) demonstrates that students successfully used the CodeDive platform for their assignment work without significant technical barriers, indicating the system’s successful integration into the course workflow. Table 3 summarizes the aggregate results: 24,845 snapshots (16,747,487 bytes), 2194 builds (1618 successful, 576 failed), and 2124 executions (2011 successful, 113 failed). These metrics demonstrate the scale of coding activity captured by CodeDive. Over the roughly two-week period, the average student created about 261 snapshots ($\approx 16,747,487/95$ bytes per student), attempted around 23 builds, and executed their program about 22 times. The build success rate was approximately 74% (1618/2194 successful), indicating that roughly one in four compilations resulted in an error that students had to debug. In contrast, the execution success rate was approximately 95% (2011/2124), suggesting that once the code compiled, it almost always ran without crashing or major runtime errors. This disparity between compilation and execution failure rates is typical in programming assignments, underscoring that compiler errors were the primary hurdle for most students.

Table 3. Assignment 1: summary of snapshot, build, and execution statistics.

Metric	Snapshot	Build	Execution
Active Students	95	95	95
Total Attempts	24,845	2194	2124
Successful Attempts	—	1618	2011
Failed Attempts	—	576	113
Total Data Size (bytes)	16,747,487	—	—

6.4. Stability and Overhead

CodeDive operated continuously without significant data loss. The pre-deployment stress tests uncovered no event omissions. Early in the semester, an `inotify watch-limit` was briefly exceeded—resulting in a small window of missed snapshot events—but no further issues occurred after raising the limit. In parallel, CodeDive monitored another course (about 200 WebIDE containers) without degradation. User-space collectors consumed on

average 0.5% and 3% of a single CPU core and 100 MiB and 300 MiB of RAM, respectively, imposing a low overhead. Although kernel-resident eBPF tracer usage cannot be measured precisely from user space, no detection delays or system overloads were observed. The eBPF-based approach was selected over alternatives like ptrace due to its superior performance, which stems from executing sandboxed code within the kernel and avoiding the high context-switching overhead associated with tracing individual system calls. The alignment between collected data volumes, daily distributions, and expected student workflows confirms reliable lossless capture. In other words, CodeDive's monitoring was able to run continuously alongside student activities without adversely affecting performance or user experience. The only minor glitch encountered (an `inotify` watch-limit exhaustion) was promptly resolved by increasing the limit, after which no further events were lost.

The fact that CodeDive simultaneously tracked around 200 containers in another course with no degradation demonstrates its stability under a significant real-world load. It is important to note that this reflects the practical scale of the deployed courses rather than an architectural limit. Because CodeDive is built on Kubernetes, it is designed to scale smoothly to thousands of users, limited only by the underlying infrastructure. As we expand its use within our university, we will continue to monitor performance at scale and report our findings.

Overall, deploying CodeDive in an active classroom provided strong evidence of its educational value and technical soundness. The system successfully captured a comprehensive log of student activities (edit, compile, and run) and converted them into meaningful metrics and visualizations. These analytics enabled instructors to monitor student engagement and identify individual patterns (or problems) in real time, going far beyond what is possible with final data submission only. At the same time, CodeDive ran reliably and efficiently in the background, introducing a low overhead and scaling to support an entire course without disruption. This experience underscores the potential of CodeDive as a practical tool to improve both teaching and learning in programming courses through data-driven insights.

6.5. Identifying Atypical Patterns

CodeDive offers educators practical tools for understanding student learning trajectories, both for providing **learning support** and for **upholding academic integrity**. Instructors can use the activity timelines to proactively identify struggling students through low activity patterns or pinpoint conceptual misunderstandings from persistent debugging errors. While this learning support is a key feature, this section details a more specific application: a practical two-stage workflow developed to address academic integrity challenges while managing instructor workload. Manually reviewing every student's activity log is infeasible in large courses. Thus, we implemented a semi-automated process that first quantitatively screens for students with anomalous development patterns and then enables a targeted qualitative review. This approach transforms CodeDive into an efficient decision-support tool, demonstrating its effectiveness through a real case study. The first stage is a data-driven screening process designed to narrow the pool of students requiring manual review. We hypothesize that students who engage in dishonest behavior like copying code will exhibit significantly less iterative development than their peers. For "Assignment 4," the average student recorded 37.96 successful builds, 10.87 failed builds, and 49.74 successful executions, clearly establishing a robust trial-and-error process baseline. Based on this, we can establish a screening policy to flag students whose activity levels fall into the bottom 25th percentile across these key metrics. This systematic screening demonstrably reduces the manual review burden; for instance, in a class with 100 students, this enables TAs to focus their efforts on a high-priority group of 12 students instead of the

entire class. The second stage involves the qualitative analysis of these flagged students' timelines. Figure 7 is a representative example from the high-priority screening group, whose case was later confirmed as academic dishonesty.

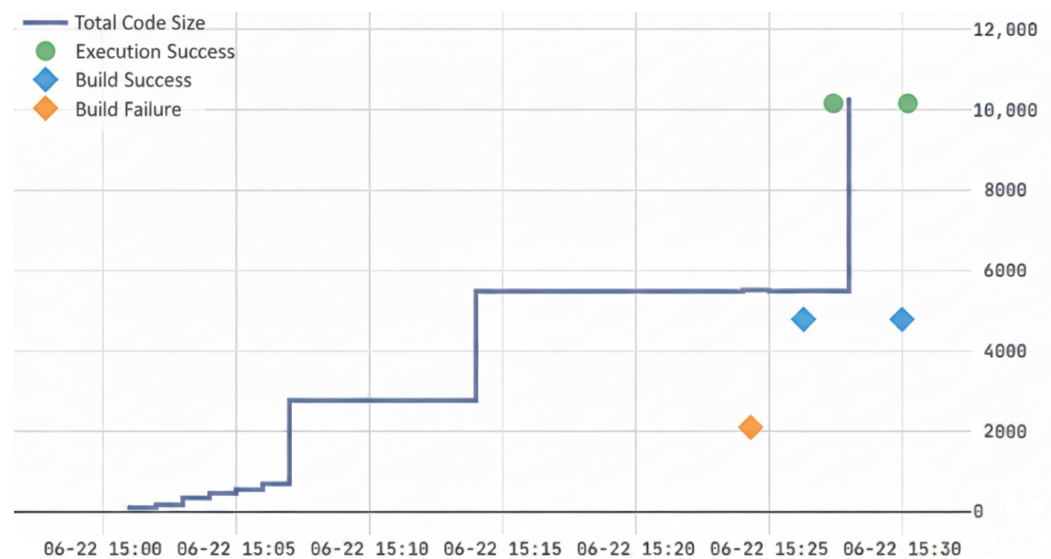


Figure 7. Activity timeline of a student confirmed to have engaged in academic dishonesty. The visualization reveals a non-incremental development process with minimal debugging effort. The solid line plots the student's source code size, orange diamonds denote build failures, blue diamonds denote successful builds, and green circles denote successful executions.

The timeline in Figure 7 reveals several clear indicators of atypical behavior:

- **Step-Function Code Growth:** The total code size does not increase gradually. Instead, it grows in large, discrete steps. This pattern strongly suggests that large chunks of pre-written code were pasted into the editor, rather than being typed out incrementally.
- **Lack of a Trial-and-Error Process:** Despite adding over 10,000 bytes of code, the student recorded only a single build failure. The development process lacks the frequent build successes, failures, and executions that characterize a genuine problem-solving effort. This starkly contrasts with the class average, indicating an absence of a typical debugging cycle.

Based on these visual anomalies, the teaching staff conducted a code review and a follow-up conversation with the student. When presented with their activity history, the student admitted to copying the code from a peer. It is important to emphasize that this workflow positions CodeDive as an assistant tool for instructors, not an automated judge. The statistical thresholds and screening policies are decision-support mechanisms to guide pedagogical assessment. The optimal application of these tools may vary depending on the course and the nature of each assignment. Future work will involve analyzing data and confirmed cases from multiple classes to develop more robust data-driven guidelines. The goal is to refine these screening methods and enhance the system's ability to support fair and efficient evaluation.

7. Discussion

7.1. Directions for Pedagogical Evaluation

This initial deployment of CodeDive successfully validated its technical architecture and demonstrated its potential to enhance faculty transparency. The rich and fine-grained data that spans code snapshots, builds, and executions provides a robust foundation for analyzing the nuances of student learning processes. In fact, the system already offers

significant insight into coding habits, debugging proficiency, error patterns, and overall development efforts, which are crucial precursors to understanding the more profound pedagogical impact. This process-oriented foundation allows the research community to move beyond technical validation and ask more profound questions about educational effectiveness. Although the current study confirms that atypical patterns can be identified, the next logical step is systematically investigating the system’s influence on specific learning outcomes. Key research questions that emerge include the following. How does real-time monitoring affect students’ final code quality? Does it lead to a measurable reduction in error frequency over time? Can we quantify its impact on promoting academic honesty? Answering these questions is essential for realizing the full potential of such monitoring tools. A more rigorous multifaceted experimental design is imperative to address these critical questions. A definitive assessment of CodeDive’s impact would require a comparative study with a control group to isolate the system’s effects on student performance. This quantitative analysis should be complemented by qualitative data from student surveys and interviews to understand the student experience, balancing skill acquisition goals with potential student anxiety. Finally, a longitudinal analysis across multiple semesters would be invaluable for identifying broader trends. This comprehensive approach is necessary to provide the concrete evidence to translate process-oriented data into actual educational value.

7.2. User Data Security and Ethical Considerations

While CodeDive incorporates robust technical security measures such as container isolation and strong authentication, the collection of fine-grained behavioral data necessitates a comprehensive ethical and data protection framework. The deployment of such a system in educational institutions requires careful consideration of international standards like the **General Data Protection Regulation (GDPR)** and national regulations such as the **Family Educational Rights and Privacy Act (FERPA)** in the United States. These regulations establish critical principles that must guide our future work, as summarized in Table 4.

Table 4. Key data protection principles and their implications for CodeDive.

Principle	Implication for CodeDive
Informed Consent	Before using the system, all users (students and instructors) must be explicitly informed about what data are collected, why they are collected, and how they will be used. They must provide clear affirmative consent.
User Rights (Access and Deletion)	The system must provide a mechanism for students to access a complete record of their collected data. Furthermore, a clear procedure must be in place for users to request the rectification or deletion of their personal data (the “right to be forgotten”).
Access Control (Legitimate Interest)	A formal policy, guided by FERPA’s principle of “legitimate educational interest,” must be established. This policy will strictly define which roles (e.g., instructors, TAs, administrators) can access what level of data and only for specific approved pedagogical purposes such as grading and feedback.
Data Minimization and Purpose Limitation	The system should be designed to collect only the minimum data necessary for its stated educational goals. These data must not be used for any purpose beyond what was originally specified to the user.

As we move forward, developing these policies and conducting a formal compliance audit will be a top priority. Our goal is to ensure that CodeDive is not only a powerful pedagogical tool but also an ethically responsible and legally compliant platform.

7.3. Resource Considerations and Architectural Trade-Offs

As detailed in Section 3.1, the CodeDive architecture was fundamentally designed to prioritize security, isolation, and a reproducible learning environment for each student. By providing a dedicated container-based WebIDE instance per user, we ensure that students' activities are sandboxed, which is paramount for upholding academic integrity in the current educational landscape. However, this robust architectural choice introduces an inherent trade-off. While our deployment evaluation in Section 6.4 confirmed a low operational overhead for individual containers, the model of provisioning one WebIDE instance per student is, by design, less memory-efficient than multi-tenant architectures where users share a common environment. This approach can lead to significant resource demands during peak usage, a critical consideration for institutions with limited infrastructure. Quantifying the precise financial and resource costs of deploying CodeDive is inherently complex, as such metrics are highly dependent on the specific infrastructural context. Factors such as choosing between a private or public cloud, institutional pricing agreements, and administrative overhead can dramatically alter cost projections. Consequently, this paper focuses on a qualitative discussion of the architectural trade-offs, an approach that provides more broadly applicable and generalizable insights for institutions considering such platforms. The system's Kubernetes-based architecture is intentionally designed to mitigate these efficiency challenges through dynamic resource allocation and auto-scaling. Nevertheless, developing a formal framework for cost-benefit analysis tailored to educational contexts presents a valuable direction for future research, offering a practical guide for effective adoption.

8. Conclusions

This paper has presented CodeDive, a cloud-based programming platform that integrates a fully functional web IDE with real-time code activity monitoring. By providing isolated containerized workspaces for each student, CodeDive eliminates environment setup issues and ensures security, while the eWatcher module continuously captures fine-grained snapshots of code edits, compilations, and executions. These features provide instructors with unparalleled insight into the learning process, allowing them to observe how students develop solutions step by step and helping to ensure academic integrity in the era of AI-assisted coding. In summary, CodeDive addresses the twin challenges of uniform development environments and detailed process tracking in programming education, providing a scalable and user-friendly system that benefits both students and teachers.

Our deployment experiences in an undergraduate Operating Systems course demonstrate the effectiveness and stability of CodeDive. During a two-week assignment with approximately 100 students, the system reliably recorded nearly 25,000 code snapshots and over 4000 compilation and execution events with minimal overhead and no disruption to users. The collected data revealed each student's coding patterns—for example, whether they worked steadily or made last-minute changes—and these insights were visualized through an interactive dashboard. Instructors were able to monitor student progress in real time, quickly identifying those who were struggling or who had suddenly pasted a large volume of code right before the deadline. This level of transparency not only facilitated timely support for students requiring assistance but also provided evidence for verifying the authenticity of students' work in the face of potential LLM misuse or other forms of academic dishonesty when irregular activity was detected.

Moving forward, we plan to expand CodeDive in several key directions. First, we will conduct a formal sensitivity analysis on system parameters, such as the snapshot threshold, by reviewing related studies and analyzing our collected data to further optimize performance and data granularity. Second, we will perform large-scale stress tests to quantitatively validate the system's operational limits and conduct performance benchmarks comparing our resource usage against industry standards to formally assess its efficiency. Third, we will conduct rigorous pedagogical studies with control groups to identify and validate patterns associated with specific learning outcomes. These validated patterns will then serve as the foundation for a suite of automated features. Specifically, we plan to develop alerts, comparative reports, and tailored recommendations for instructors. These additions are designed to reduce instructor workload and fully transform CodeDive into an effective decision-support system. Finally, to enhance its versatility as a more general-purpose platform, we will extend the architecture to support other educational domains, with immediate plans to integrate monitoring for Jupyter Notebook (<https://github.com/jupyter/notebook> (accessed on 22 September 2025)) interactions for data science courses and Git-based workflows for team-based software engineering projects.

Author Contributions: Conceptualization, H.P., Y.K. and K.L.; Methodology, H.P., Y.K. and K.L.; Software, H.P., S.J., J.K., Y.H., G.K. and E.K.; Validation, H.P., Y.K. and K.L.; Investigation, H.P., Y.K., K.L., S.J., J.K., Y.H., G.K. and E.K.; Resources, H.P.; Data curation, H.P., S.J., J.K., Y.H., G.K. and E.K.; Writing—original draft preparation, H.P., Y.K., K.L., S.J., J.K., Y.H., G.K. and E.K.; Writing—review and editing, H.P., Y.K. and K.L.; Visualization, H.P., Y.K. and K.L.; Supervision, H.P.; Project administration, H.P.; Funding acquisition, H.P., Y.K. and K.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the MSIT (Ministry of Science, ICT), Korea, under the National Program for Excellence in SW), supervised by the IITP (Institute of Information & communications Technology Planning & Evaluation) in 2025 (2022-0-01067) and by the research funds of Jeonbuk National University in 2022 and by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2022-00166222) and by the IITP-ITRC (Information Technology Research Center) grant funded by the Korean government (MSIT, IITP-2025-RS-2023-00259061).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All data supporting the findings of this study are contained within the published article. Additional data sets that may include personal or sensitive information on individual students can be made available by the corresponding authors upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

IDE	Integrated Development Environment
UI	User Interface
eBPF	Extended Berkeley Packet Filter
REST	Representational State Transfer
API	Application Programming Interface
VS Code	Visual Studio Code
SSH	Secure Shell
JSON	JavaScript Object Notation

References

1. Bergström, J.J.J.; Repo, V.E.; Tuunainen, V.K. Challenges to academic integrity from new tools: A survey of students' perceptions and behaviors of employing ChatGPT. In Proceedings of the Annual Hawaii International Conference on System Sciences. Hawaii International Conference on System Sciences, Honolulu, HI, USA, 3–6 January 2024; pp. 74–83.
2. Lee, V.R.; Pope, D.; Miles, S.; Zárate, R.C. Cheating in the Age of Generative AI: A High School Survey Study of Cheating Behaviors before and after the Release of ChatGPT. *Comput. Educ. Artif. Intell.* **2024**, *7*, 100253. [\[CrossRef\]](#)
3. Khalil, M.; Er, E. Will chatgpt get you caught? rethinking of plagiarism detection. In Proceedings of the International Conference on Human-Computer Interaction, Copenhagen, Denmark, 23–28 July 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 475–487.
4. Han, S.; Zhang, Z.; Liu, H.; Kong, W.; Xue, Z.; Cao, T.; Shu, J. From engagement to performance: The role of effort regulation in higher education online learning. *Interact. Learn. Environ.* **2024**, *32*, 6607–6627. [\[CrossRef\]](#)
5. Yang, B.; Xie, C.; Liu, T.; Xu, J.; Li, W. Exploring the relationship between teacher talk supports and student engagement from the perspective of students' perceived care. *Interact. Learn. Environ.* **2024**, *32*, 6393–6412. [\[CrossRef\]](#)
6. Zhang, Z.; Zhang, E.; Liu, H.; Han, S. Examining the association between discussion strategies and learners' critical thinking in asynchronous online discussion. *Think. Skills Creat.* **2024**, *53*, 101588. [\[CrossRef\]](#)
7. Liu, T.; Liu, H.; Yang, B.; Zhang, Z. LDCNet: Limb direction cues-aware network for flexible HPE in industrial behavioral biometrics systems. *IEEE Trans. Ind. Inform.* **2023**, *20*, 8068–8078. [\[CrossRef\]](#)
8. Liu, H.; Liu, T.; Chen, Y.; Zhang, Z.; Li, Y.F. EHPE: Skeleton cues-based gaussian coordinate encoding for efficient human pose estimation. *IEEE Trans. Multimed.* **2022**, *26*, 8464–8475. [\[CrossRef\]](#)
9. Liu, H.; Liu, T.; Zhang, Z.; Sangaiah, A.K.; Yang, B.; Li, Y. ARHPE: Asymmetric relation-aware representation learning for head pose estimation in industrial human–computer interaction. *IEEE Trans. Ind. Inform.* **2022**, *18*, 7107–7117. [\[CrossRef\]](#)
10. Brown, N.C.C.; Altadmri, A.; Sentance, S.; Kölling, M. Blackbox, Five Years On: An Evaluation of a Large-Scale Programming Data Collection Project. In Proceedings of the 2018 ACM Conference on International Computing Education Research (ICER '18), Espoo, Finland, 13–15 August 2018; ACM: New York, NY, USA, 2018; pp. 196–204.
11. Lyulina, E.; Birillo, A.; Kovalenko, V.; Bryksin, T. TaskTracker-tool: A Toolkit for Tracking of Code Snapshots and Activity Data During Solution of Programming Tasks. In Proceedings of the 52nd ACM Technical Symposium on Computer Science Education (SIGCSE '21), Virtual, 13–20 March 2023; ACM: New York, NY, USA, 2021; pp. 495–501.
12. Cao, Y.; Wang, L.; Zheng, Z.; Tao, X. A Tool for Non-Intrusive and Privacy-Preserving Developers' Programming Activity Data Collection. In Proceedings of the 2020 IEEE 44th Annual Computers, Software and Applications Conference (COMPSAC), Madrid, Spain, 13–17 July 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 953–962. *DevActRec*.
13. Price, T.W.; Hovemeyer, D.; Rivers, K.; Gao, G.; Bart, A.C.; Kazerooni, A.M.; Becker, B.A.; Petersen, A.; Gusukuma, L.; Edwards, S.H.; et al. ProgSnap2: A Flexible Format for Programming Process Data. In Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20), Trondheim, Norway, 15–19 June 2020; ACM: New York, NY, USA, 2020; pp. 356–362.
14. Kim, Y.; Lee, K.; Park, H. Watcher: Cloud-Based Coding Activity Tracker for Fair Evaluation of Programming Assignments. *Sensors* **2022**, *22*, 7284. [\[CrossRef\]](#) [\[PubMed\]](#)
15. Birillo, A.; Tigina, M.; Kurbatova, Z.; Potriasaeva, A.; Vlasov, I.; Ovchinnikov, V.; Gerasimov, I. Bridging Education and Development: IDEs as Interactive Learning Platforms. In Proceedings of the IDE @ICSE 2024 Workshop, Lisbon, Portugal, 20 April 2024; ACM: New York, NY, USA, 2024; pp. 53–58.
16. The Kubernetes Authors. Kubernetes: Production-Grade Container Scheduling and Management. Latest Release v1.34.1 (as of 9 September 2025). 2025. Available online: <https://kubernetes.io/> (accessed on 27 June 2025).
17. McCanne, S.; Jacobson, V. The BSD Packet Filter: A New Architecture for User-Level Packet Capture. In Proceedings of the USENIX Winter 1993 Conference, San Diego, CA, USA, 25–29 January 1993; USENIX Association: Berkeley, CA, USA, 1993.
18. Habibi, B.; Nakanishi, T.; Fukuda, A. Student's Programming Activity Tracking System to Help Instructors of the Programming Exercise. In Proceedings of the 2016 IEEE Region 10 Symposium (TENSYP), Bali, Indonesia, 9–11 May 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 89–94.
19. Sölch, M.; Dietrich, F.T.J.; Krusche, S. Direct Automated Feedback Delivery for Student Submissions based on LLMs. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE '25), Trondheim, Norway, 23–28 June 2025; ACM: New York, NY, USA, 2025; pp. 901–911.
20. Brown, A.; Green, B. CodeInsights: Monitoring Programming Students' Progress. In Proceedings of the 17th International Conference on Computer Systems and Technologies (CompSysTech '16), Palermo, Italy, 23–24 June 2016; pp. 375–382.
21. Chen, L.; Kumar, R. TrackThinkDashboard: Understanding Student Self-Regulated Learning Workflows. *arXiv* **2025**, arXiv:2503.19460.
22. Garcia, M.; Lee, S. Design of an Online Programming Platform and a Study on Learners' Test-Case Behaviors. *Electronics* **2023**, *12*, 4596.

23. Patel, R.; Wong, L. Student Adoption and Perceptions of a Web Integrated Development Environment. In Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20), Portland, OR, USA, 11–14 March 2020; pp. 1172–1178.
24. Anderson, P.; Thomas, K. Real-Time Collaborative Programming in Undergraduate Education. *J. Educ. Technol.* **2024**, *35*, 78–88.
25. Baker, R.S.; Brown, B.C.D. Leveraging the Integrated Development Environment for Learning Analytics. In *The Cambridge Handbook of Computing Education Research*; Fincher, S.A., Robins, A.V., Eds.; Cambridge University Press: Cambridge, UK, 2019; pp. 563–592.
26. The Helm Authors. Helm: The Kubernetes Package Manager. Latest Release v3.16.4; Graduated CNCF Project. 2025. Available online: <https://helm.sh/> (accessed on 27 June 2025).
27. inotify Authors. Inotify(7): Linux Kernel File Change Notification. Linux Manual Page. 2025. Available online: <http://man7.org/linux/man-pages/man7/inotify.7.html> (accessed on 27 June 2025).
28. Watchdog Authors. watchdog 2.1.5. PyPI Package for File System Event Monitoring in Python. 2025. Available online: <https://pypi.org/project/watchdog/> (accessed on 27 June 2025).
29. Keycloak Authors. Keycloak: Open Source Identity and Access Management. Latest Release 26.2.5; CNCF Incubating Project. 2025. Available online: <https://www.keycloak.org/> (accessed on 27 June 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.